

## รายงานโครงงาน Object oriented data structure

### แนวโลกป็นท้องฟ้าเป็นต่อ

โดย

67010500 นายบดินทร์ สันทวีเลิศ

67010585 นายเป็นต่อ เฮงหลี

67010686 นายภักพล เพ็ญศิริภักดี

67011031 นางสาวอลิสรา วาสิลิกี้ แคทซูลิส

67011569 นายสุพศิน รัตนพันธ์

เสนอ

ผศ. เกียรติณรงค์ ทองประเสริฐ

โครงงานนี้เป็นส่วนหนึ่งของการศึกษา

วิชา 01076109 OBJECT ORIENTED DATA STRUCTURES

ปีการศึกษา 2568

|             |                                |                       |
|-------------|--------------------------------|-----------------------|
| ชื่อโครงการ | แมวโลกปิ่นทองฟ้าเป็นต่อ        |                       |
| ผู้จัดทำ    | นายบดินทร์ สิ้นทวีเลิศ         | รหัสนักศึกษา 67010500 |
|             | นายเป็นต่อ เฮงหลี              | รหัสนักศึกษา 67010585 |
|             | นายภักพล เพ็ญศิริภักดิ์        | รหัสนักศึกษา 67010686 |
|             | นางสาวอลิสา วาสิลิกี้ แคทซูลิส | รหัสนักศึกษา 67011031 |
|             | นายสุพศิน รัตนพันธ์            | รหัสนักศึกษา 67011569 |

### บทคัดย่อ

โครงการนี้มีวัตถุประสงค์เพื่อพัฒนาโปรแกรมจำลองการจัดการห้องพักในโรงแรม โดยประยุกต์แนวคิดเชิงวัตถุ (Object-Oriented Programming) และโครงสร้างข้อมูลแบบผสม ได้แก่ Treap Tree และ Dictionary (Hashed-based) เพื่อเพิ่มประสิทธิภาพในการจัดเก็บและค้นหาข้อมูลห้องพัก โปรแกรมสามารถเพิ่ม ลบ แสดงผล ค้นหา และจัดเรียงหมายเลขห้องได้อย่างรวดเร็ว รวมถึงสามารถบันทึกข้อมูลแชกลงในไฟล์ Excel เพื่อเก็บบันทึกข้อมูลภายนอกได้อีกด้วย ผลการพัฒนาแสดงให้เห็นว่าการผสมการทำงานของ Treap Tree และ Dictionary สามารถลดเวลาเฉลี่ยในการค้นหาและจัดการข้อมูลได้อย่างมีประสิทธิภาพและมีความยืดหยุ่นสูง เหมาะสำหรับการประยุกต์ใช้กับระบบจัดการข้อมูลที่ต้องการความเร็วและการเข้าถึงข้อมูลบ่อยครั้ง

## 1. คำนำ

รายงานฉบับนี้เป็นส่วนหนึ่งของรายวิชา 01076109 Object-Oriented Data Structures โดยมีจุดมุ่งหมายเพื่อศึกษาการนำหลักการเขียนโปรแกรมเชิงวัตถุ (OOP) มาประยุกต์ใช้ในการออกแบบและพัฒนาโครงสร้างข้อมูลเชิงซ้อนในรูปแบบโปรแกรมจริง โครงการนี้ได้เลือกใช้ Treap Tree เพื่อจัดเรียงข้อมูลแบบมีสมมูลโดยอัตโนมัติ และ Dictionary เพื่อจัดเก็บและค้นหาข้อมูลได้อย่างรวดเร็ว ทั้งสองโครงสร้างถูกออกแบบให้ทำงานร่วมกันภายใต้คลาสหลัก “Hotel” ซึ่งเป็นศูนย์กลางในการควบคุมระบบการจัดการห้องพักทั้งหมด

ผู้จัดทำหวังว่าโครงการนี้จะเป็แนวทางให้ผู้สนใจสามารถเข้าใจแนวคิดของการใช้โครงสร้างข้อมูลขั้นสูงในการแก้ปัญหาลจริง และต่อยอดไปสู่การพัฒนาาระบบในระดับที่ซับซ้อนมากยิ่งขึ้น

## 2. วิธีการใช้งานโปรแกรม

เมื่อเปิดใช้งานโปรแกรม จะขึ้นให้กรอกจำนวนแขกในรอบแรก

เริ่มต้นใช้หน่วยความจำ: 75.46 MB  
Enter guest first group :

โดยในการกรอกสามารถกรอกในรูปแบบ : {วิธีการเดินทาง} {จำนวน}/.../w {จำนวนคน}, ...

เช่น

Enter guest first group : Bus 1/Train 2/w 100, w 20

ผลลัพธ์คือ เพิ่มแขกจำนวน 120 คน

เมื่อกรอกแขกชุดแรกไปแล้ว โปรแกรมจะขึ้นให้เลือกใช้ฟังก์ชันต่อไป โดยมีตัวเลือกดังนี้ :

- (1) Add new guest : เพิ่มแขกใหม่ การทำงานแบบเดียวกับการกรอกจำนวนแขกในรอบแรก
- (2) Add manual room : เพิ่มห้อง โดยเลือกกรอกเป็นเลขห้องที่ต้องการให้แขกอยู่เลย หากห้องนั้นมีแขกอยู่แล้ว สามารถเลือกได้ว่าจะไม่เพิ่มห้องหรือจะเพิ่มเข้าไปโดยจัดเรียงแขกเก่าก่อน
- (3) Delete a room : ลบห้องออก
- (4) Show all room numbers (sorted) : แสดงห้องทั้งหมดแบบเรียงเลข
- (5) Search guest by room number : ค้นหาห้องด้วยเลขห้อง
- (6) Save data to file : ส่งออกไฟล์ .xlsx ที่มีข้อมูล
- (7) Get memory used : แสดง memory ที่ใช้
- (0) Exit : ออกจากโปรแกรม

### 3. Class Overview

หัวข้อนี้จะกล่าวถึงแต่ละคลาสโดยสังเขป และความสัมพันธ์ระหว่างคลาส

#### 3.1. Treap Tree

คลาสนี้ เก็บหมายเลขห้อง และเรียงข้อมูลตอนเรียกใช้ เราเลือกใช้ Treap Tree เนื่องจาก Treap Tree มีความสามารถในการจัดเรียงข้อมูลตั้งแต่เริ่มอยู่แล้ว ทำให้อ่านแบบ InOrder ทำให้ได้ตัวเลขที่เรียงจากมากไปน้อยเสมอ ซึ่งมากลบจุดอ่อนของ Hash Table ที่มีประสิทธิภาพในการเรียงข้อมูลต่ำ

#### 3.2. Dictionary

เราเลือกใช้ Python Dictionary เป็นโครงสร้างข้อมูลหลักในการจัดเก็บข้อมูลห้องพัก โดย Dictionary ของ Python ใช้หลักการ Hash Table ภายใน ซึ่งให้ประสิทธิภาพในการค้นหาและเพิ่มข้อมูลเฉลี่ย  $O(1)$

คลาส Hash\_Table ของเราทำหน้าที่เป็น wrapper class สำหรับ Python dict โดยเก็บข้อมูล key (เลขห้องแขก) และ values (สถานะแขก) โดย 0 แทนผู้เข้าพักเก่า, 1 แทนผู้เข้าพักใหม่, และ "Manual Add" สำหรับห้องที่เพิ่มด้วยตนเอง

Python Dictionary จัดการ collision และ resizing โดยอัตโนมัติภายใน ทำให้โครงสร้างมีความเสถียรและประสิทธิภาพสูง โดยทำงานร่วมกับ Treap Tree เพื่อจัดการข้อมูลอย่างมีประสิทธิภาพ

#### 3.3. Hotel

คลาสนี้เป็นศูนย์กลางในการควบคุมโปรแกรมทั้งหมด ตั้งแต่การเพิ่มแขกเข้า ทั้งแบบธรรมดา และ manual ลบออก แสดงเลขห้องทั้งหมด ค้นหาแขกด้วยเลขห้อง และส่งออกเป็นไฟล์ .xlsx โดยเวลาจัดการข้อมูล จะทำทั้งใน Treap Tree และ ใน Hash Table

#### Class TreapTree

```
class TreapTree:
    class Node:
        __slots__ = ('data', 'priority', 'left', 'right')
        def __init__(self, data):
            self.data = data
            self.priority = random.random()
            self.left = None
            self.right = None

        def __str__(self):
            return f"{self.data}({self.priority:.3f})"

    def __init__(self):
        self.root = None
```

#### Class HashTable

```
class Hash_Table:
    def __init__(self):
        self.table = {}
```

#### Class Hotel

```
class Hotel:
    def __init__(self):
        self.TreapTree = TreapTree()
        self.Hash_table = Hash_Table()
        self.prime_list = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
```

## 4. ฟังก์ชันในคลาสต่าง ๆ ของโปรแกรม

### 4.1. TreapTree

#### 4.1.1. add

```
def add(self, root, data):
    self.root = self._add(root, data)
    return self.root

def _add(self, root, data):
    if root is None:
        return self.Node(data)
    if data < root.data:
        root.left = self._add(root.left, data)
        if root.left.priority > root.priority:
            root = self._rotateRight(root)
    elif data > root.data:
        root.right = self._add(root.right, data)
        if root.right.priority > root.priority:
            root = self._rotateLeft(root)
    return root
```

ฟังก์ชันนี้เพิ่มโหนด/เลขห้องใหม่เข้า Treap โดยแทรกตามกฎ BST ด้วย data แล้วปรับสมดุลด้วยการ เช็ค priority และ หมุน (rotate) เพื่อคง heap property ไว้ ผลคือได้ต้นไม้ที่สมดุลโดยคาดหมาย

#### 4.1.2. \_rotateRight / \_rotateLeft

```
def _rotateRight(self, y):
    x = y.left
    y.left = x.right
    x.right = y
    return x

def _rotateLeft(self, x):
    y = x.right
    x.right = y.left
    y.left = x
    return y
```

ฟังก์ชันนี้ถูกใช้ใน add เพื่อคงสมดุลของ tree หาก ลูกขวา priority สูงกว่า หมุนซ้าย หาก ลูกซ้าย priority ต่ำกว่า หมุนขวา

## 4.1.3. delete

```

def delete(self, data):
    self.root, deleted = self._delete(self.root, data)
    return deleted

def _delete(self, root, data):
    if root is None:
        return None, False

    deleted = False
    if data < root.data:
        root.left, deleted = self._delete(root.left, data)
    elif data > root.data:
        root.right, deleted = self._delete(root.right, data)
    else:
        deleted = True
        # หมุนลงไปจนกลายเป็นใบ แล้วลบทิ้ง
        if root.left is None and root.right is None:
            return None, True
        elif root.left is None:
            root = self._rotateLeft(root)
            root.left, _ = self._delete(root.left, data)
        elif root.right is None:
            root = self._rotateRight(root)
            root.right, _ = self._delete(root.right, data)
        else:
            # หมุนฝั่งที่ priority สูงกว่า
            if root.left.priority > root.right.priority:
                root = self._rotateRight(root)
                root.right, _ = self._delete(root.right, data)
            else:
                root = self._rotateLeft(root)
                root.left, _ = self._delete(root.left, data)
    return root, deleted

```

ฟังก์ชันนี้ใช้สำหรับ ลบโหนดออกจาก Treap Tree โดยยังคง สมบัติของ BST (ซ้าย < root < ขวา) และ สมบัติของ Heap (priority ของพ่อ > ลูก) โดยมีกรณีที่เป็นไปได้ดังนี้

root = None → ลบไม่สำเร็จ

ไม่มีลูก → ลบได้เลย

มีลูกเดียว → เอาลูกนั้นขึ้นแทนที่

มีลูกสองคน → หมุนฝั่งที่ priority สูงกว่า

#### 4.1.4. Inorder

```
def InOrder(self):  
    result = []  
    self._inorder(self.root, result)  
    return result  
  
def _inorder(self, node, result):  
    if node:  
        self._inorder(node.left, result)  
        result.append(node)  
        self._inorder(node.right, result)
```

ฟังก์ชันนี้ใช้สำหรับ แสดงผลข้อมูลใน Treap Tree ตามลำดับเรียงจากน้อยไปมาก  
โดยใช้หลักการ Inorder Traversal คือ (ซ้าย → ราก → ขวา)

## 4.2. Hash\_Table

### 4.2.1. \_\_init\_\_()

```
class Hash_Table:
    def __init__(self):
        self.table = {}
```

ฟังก์ชันนี้สร้าง instance ของคลาสโดยกำหนดให้ self.table เป็น dictionary เปล่า ({} สำหรับเก็บข้อมูลห้องพัก

### 4.2.2. Insert (key, value)

```
def insert(self, key, value):
    self.table[key] = value
```

ฟังก์ชันนี้ใช้สำหรับแทรกหรืออัปเดตข้อมูลลงใน dictionary โดยใช้ key เป็นหมายเลขห้อง และ value เป็นสถานะของแขก (0, 1, หรือ "Manual Add")

เวลาในการทำงาน:  $O(1)$  เฉลี่ย เนื่องจาก Python dict ใช้ hash table ภายใน

### 4.2.3. get(key)

```
def get(self, key):
    return self.table.get(key)
```

ฟังก์ชันนี้ใช้สำหรับค้นหาค่าที่ตรงกับ key ที่ระบุ โดยจะคืนค่า value หากพบ หรือ None หากไม่พบ

เวลาในการทำงาน:  $O(1)$  เฉลี่ย

#### 4.2.4. delete(key)

```
def delete(self, key):
    return self.table.pop(key, None) is not None
```

ฟังก์ชันนี้ใช้สำหรับลบข้อมูลออกจาก dictionary โดยใช้เมธอด pop() ซึ่งจะคืนค่า True หากลบสำเร็จ หรือ False หากไม่พบ key  
เวลาในการทำงาน:  $O(1)$  เฉลี่ย

### 4.3. Hotel

#### 4.3.1. add\_room\_manual (requirement function)

```
def add_room_manual(self, room_number: int):
    if self.Hash_table.get(room_number) is None:
        self.Hash_table.insert(room_number, "Manual Add")
        self.TreapTree.add(self.TreapTree.root, room_number)
        print(f"✅ Room {room_number} added successfully.")
        return room_number
    print(f"⚠️ Room {room_number} already exists: {self.Hash_table.get(room_number)}")
    while True:
        choice = input(
            "Collision detected. Choose an action:\n"
            "  [S]kip : Do not add this room\n"
            "  [M]ove : Reassign ALL existing guests, then place THIS guest here\n"
            "  ⭐ Enter S or M: "
        ).strip().lower()
        if choice in ("s", "skip"):
            print("❌ Skipped adding this room.")
            return None
        if choice in ("m", "move"):
            print("🔄 Reassigning all existing guests to new room numbers...")
            if self.Hash_table.get(room_number) is not None:
                prev = self.Hash_table.get(room_number)
                if prev == 0:
                    print_guest_info = "Old Guest"
                elif prev == 1:
                    print_guest_info = "New Guest"
                else:
                    print_guest_info = "Manual Add"
                self.Hash_table.delete(room_number)
                self.TreapTree.root, _ = self.TreapTree.delete(self.TreapTree.root, room_number)
                print(f"🗑️ Force-freed room {room_number} (previous occupant: {room_number} : {print_guest_info}).")
                self.Hash_table.insert(room_number, "Manual Add")
                self.TreapTree.add(self.TreapTree.root, room_number)
                print(f"✅ Room {room_number} assigned to the new manual guest after reassignment.")
                return room_number
            print("❗ Invalid input. Please type only 'S' or 'M'.")
```

ฟังก์ชันนี้ทำหน้าที่ เช็คว่าถ้าห้องยังไม่มีอยู่ใน Hash\_table ให้ insert ลง Hash แล้ว add ใน Treap Tree เพื่อรักษาลำดับห้อง ถ้าห้องซ้ำจะถามผู้ใช้ว่าให้ Skip หรือ Move (ย้ายข้อมูลเดิมก่อนแล้วค่อยแทรก)

เวลาในการทำงาน :

- **Best Case  $O(1)$ :** ห้องซ้ำและผู้ใช้เลือก skip ทันที → ตรวจสอบ Hash เพียงครั้งเดียว
- **Average Case  $O(\log n)$ :** ห้องไม่ซ้ำ → Hash get/insert  $O(1)$  + Treap add  $O(\log n)$
- **Worst Case  $O(n)$ :** Hash table เกิด collision ร้ายแรง → get/insert degrade เป็น  $O(n)$  + Treap add  $O(\log n)$  ซึ่งเกิดขึ้นได้ยากมาก เพราะ dict ใช้ hash, resize และ probing ดีมาก หรือ Treaptree กรณี quality ของ random priority ไม่ดีและ node หนักไปทางข้างใดข้างหนึ่งเสมือนใช้ Linklist

#### 4.3.2. delete\_room\_manual (requirement function)

```
def delete_room_manual(self, room_number : int):
    deleted_hash = self.Hash_table.delete(room_number)
    deleted_tree = self.TreapTree.delete(room_number)
    if deleted_hash or deleted_tree:
        return room_number
    else:
        return f"Not found room {room_number}"
```

ฟังก์ชันนี้ทำหน้าที่ลบหมายเลขห้องที่ระบุออกจากทั้ง Hash และ TreapTree

ขั้นตอนหลักคือ เช็คค่า หมายเลขห้อง มีอยู่จริงใน Hash\_table หรือไม่ ถ้ามีให้ลบออกจาก Hash\_table และ Treaptree

เวลาในการทำงาน :

##### Best Case – $O(1)$

- ห้องไม่อยู่ใน Hash\_table  $\rightarrow$  get() คืนค่า None  $\rightarrow$  return ทันที
- ไม่ต้องไปแตะ TreapTree

##### Average Case – $O(\log n)$

- ห้องอยู่ใน Hash\_table  $\rightarrow$  delete ใน dict =  $O(1)$
- ลบใน TreapTree =  $O(\log n)$  (search + remove + rebalance)

##### Worst Case – $O(n)$

- ถ้า Python dict มี collision ร้ายแรงหรือ adversarial keys  $\rightarrow$  delete =  $O(n)$
- TreapTree priority เลือกไม่ดี tree degenerate กลายเป็น linked list  $\rightarrow O(n)$

#### 4.3.3. sort\_room (requirement function)

```
def sort_room(self):
    return self.AVLtree.InOrder()
```

ฟังก์ชันนี้ทำหน้าที่ Return รายการห้องทั้งหมดแบบเรียงจากน้อยไปมาก ด้วยการทำ

InOrderTraversal ของ TreapTree ซึ่งเก็บหมายเลขห้อง

เวลาในการทำงานคือ bigO:  $O(n)$  สำหรับการเดินครบทุกโหนด

#### 4.3.4. search\_room\_by\_number (requirement function)

```
def search_room_by_number(self, room_number : int):
    return self.Hash_table.get(room_number)
```

ฟังก์ชันนี้ทำหน้าที่ return value ของ key (room\_number) นั้นโดยค้นหาจากใน Hash

Table

กรณี Best case และ Average bigO:  $O(1)$

เงื่อนไขเมื่อไม่มีการชนเกิดขึ้นเลยของ key นั้นใน hash table

กรณี Worst case bigO:  $O(n)$  เมื่อเกิดการชนกับทุกข้อมูลใน Hash Table

#### 4.3.5. manage\_new\_guest

```
def manage_new_guest(self, new_guest_list):
    self.manage_current_guest()
    root = self.TreapTree.root

    for guest_str in new_guest_list:
        parts = guest_str.split('/')
        list_value = []
        walk = 0

        for part in parts:
            k, v = part.split()
            v = int(v)
            k = k.lower()
            if k == 'w':
                walk = v
            else:
                list_value.append(v)

        guest_template = tuple(list_value)

        for w in range(walk):
            guest_tuple = guest_template + (w,)
            root = self.add_new_guest(root=root, guest_info=guest_tuple)
```

ฟังก์ชันนี้สำหรับจัดการ input ที่เข้ามาและจัดการแยกแยะในการเรียกฟังก์ชัน

manage\_current\_guest และนำแขกใหม่ที่ละคนไปเพิ่มห้องให้ด้วยฟังก์ชัน add\_new\_guest

BigO :  $O(n)$  เพราะทำงานลูปตามจำนวนแขกทั้งหมดที่เข้ามาใหม่

#### 4.3.6. manage\_current\_guest และ \_manage\_current\_guest

```
def manage_current_guest(self):
    temp_hash = Hash_Table()
    count = [0]
    self._manage_current_guest(self.TreapTree.root, temp_hash, count)
    self.Hash_table = temp_hash
    print("-----")

def _manage_current_guest(self, root, temp_hash, count):
    if root:
        self._manage_current_guest(root.left, temp_hash, count)

        old_value = self.Hash_table.get(root.data)
        new_room_number = (count[0] + 1) ** 2
        root.data = new_room_number
        temp_hash.insert(new_room_number, 0)
        count[0] += 1

        self._manage_current_guest(root.right, temp_hash, count)
```

ฟังก์ชันนี้สำหรับ จัดห้องใหม่ให้กับแขกเดิมทั้งหมด โดยค้นหาเข้าไปใน Treaptree ที่ละ Node แบบ inorder จากนั้นจะกำหนดห้องใหม่ตาม ลำดับตนเองกำลัง 2

ถ้า tree มี n node, traversal เรียกแต่ละ node 1 ครั้ง  $\rightarrow O(n)$

เวลาในการทำงาน :

กรณี Best Case และ Average case bigO:  $O(n)$

กรณีจัดเรียงปกติ

กรณี Worst case bigO:  $O(n^2)$

กรณี hash table Collision เยอะ

## 4.3.7. add\_new\_guest

```
def add_new_guest(self, root, guest_info):
    room_number = self.find_room_number(guest_info)

    # ใช้ get() แทน in
    if self.Hash_table.get(room_number) is not None:
        i = 1
        new_room_number = room_number
        while self.Hash_table.get(new_room_number) is not None:
            i += 1
            new_room_number = room_number + i**2
        room_number = new_room_number

    # ใช้ insert() แทน []
    self.Hash_table.insert(room_number, 1)
    new_root = self.TreapTree.add(root, room_number)
    return new_root
```

ฟังก์ชันนี้มีไว้สำหรับรับเพิ่มแขกใหม่จากเดิมและบันทึกข้อมูลการเดินทางของแขกลงใน Hash และ Treaptree ตามเลขห้องที่ได้จากการคำนวณ โดยคำนวณเลขห้องใหม่ (room\_number) + ตรวจสอบซ้ำ

เวลาในการทำงาน :

กรณี Best case bigO:  $O(1)$

ห้องใหม่ไม่ซ้ำเลย

กรณี Average case bigO:  $O(1)$

ห้องใหม่ไม่ชนกันบ่อย

กรณี Worst case bigO:  $O(n)$

ห้องใหม่ชนกับทุกห้อง (แต่เกิดได้น้อยมาก จากสูตรหาเลขห้อง) หรือ treaptree priority เลือกไม่ดี  
เสมือน Linklist

#### 4.3.8. Find\_room\_number

```
def find_room_number(self, guest_info):
    room_number = 1
    i = 0

    for v in guest_info:
        if i == 1 and v == 0:
            v = 1
        if i >= len(self.prime_list):
            self.prime_list.append(self._next_prime(self.prime_list[i]))

        room_number *= (v + 1) ** self.prime_list[i]

        i += 1

    return room_number
```

ฟังก์ชันนี้ไว้สำหรับคำนวณเลขห้องจากพารามิเตอร์ที่เข้ามา

เวลาในการทำงาน :

$O(N)$  เนื่องจากต้องดู value ทั้งหมดใน tuple

#### 4.3.9. \_next\_prime

```
def _next_prime(n):
    # คืนค่า prime number ที่มากกว่าหรือเท่ากับ n
    def is_prime(x):
        if x < 2: return False
        for i in range(2, int(x**0.5)+1):
            if x % i == 0: return False
        return True
    while not is_prime(n):
        n += 1
    return n
```

ฟังก์ชันนี้จะรับเลขจำนวนเต็มมาและคืนค่าจำนวนเฉพาะตัวที่มากกว่าหรือเท่ากับตัวมัน

## 4.3.10. save\_to\_excel

```
def save_to_excel(self, filename="guest_list.xlsx"):
    data = []

    for node in self.TreapTree.InOrder():
        room_number = node.data
        guest_info = self.Hash_table.get(room_number)
        print_guest_info = None

        if guest_info is None:
            continue

        elif guest_info == 0:
            print_guest_info = "Old Guest"
        elif guest_info == 1:
            print_guest_info = "New Guest"
        else:
            print_guest_info = "Manual Add (after reassign)"

        data.append({
            "Room Number": room_number,
            "Guest Info": print_guest_info
        })

    df = pd.DataFrame(data)
    df.to_excel(filename, index=False)
    print(f"✅ Data saved to {filename} (raw guest_info version)")
```

ฟังก์ชันนี้ทำหน้าที่ไล่โหนดใน Treaptree เพื่อดึง guest\_info จาก Hash\_table ตามที่เก็บ  
จริงในระบบ แล้วเขียนเป็นไฟล์ excel  
เวลาในการทำงาน

**กรณี Best case และ Average case bigO:  $O(n)$**

ต้องเดิน  $n$  โหนดของ TreapTree เสมอ + Hash.get ต่อโหนดเฉลี่ย  $O(1)$

**กรณี Worst case bigO:  $O(n^2)$**

ถ้า Hash คุณภาพไม่ดี Hash.get ต่อโหนด =  $O(n)$  และทำจำนวน  $n$  ครั้ง รวมเป็น  $O(n^2)$